

Hybrid ML Web Security Gateway for SSRF and Cloud Metadata Attacks

Draft for feedback · POC status and research plan · Code: github.com/blastre/ml-web-security-gateway

Problem. Many web apps fetch a URL the user supplies (link previews, webhooks). An attacker can point that URL at the cloud VM's **metadata service**, which hands out temporary credentials, or at internal services. Filters that only look at the incoming URL miss tricks such as **redirects**, **domain names that resolve to internal IPs**, and **alternative IP formats** (for example 0xa9fea9fe is the same address as 169.254.169.254).

1. What the current POC does

A local Docker lab stands in for a cloud VM. The metadata service is fake and holds dummy credentials.



Demo request	Result
Normal public URL	✅ allowed
Direct metadata URL <code>http://169.254.169.254/...</code>	❌ blocked at gateway
Harmless-looking URL that redirects to metadata	❌ blocked at egress guard
Same attack with protection switched off	⚠️ dummy token leaked (the baseline)

- **ML:** Logistic Regression, Random Forest and Isolation Forest, trained on a **synthetic** set of 720 URLs (22 attack types, 14 benign types). Best held-out result: F1 0.80 with no false positives. This is a small sanity check, not a real-world result.
- **Agent:** Codex, read-only, produced a structured root-cause report for a blocked redirect attack.

2. How the research would proceed

Goal: move from the lab POC to a gateway tested on a **real cloud VM (GCP)**, and show whether it helps in practice.

1. **Make it deployable.** Run the egress guard as a standalone proxy on the VM, with all outbound traffic forced through it, so it protects any app and not just Python ones.
2. **Test on a real GCP VM** using intentionally vulnerable test apps and limited/dummy credentials only.
3. **Measure:**
 - attacks blocked, per technique;
 - normal requests wrongly blocked;
 - added delay per request;
 - the value of each layer (inbound only vs egress only vs both);
 - whether the agent's reports are correct and useful.
4. **Improve the data:** more realistic normal traffic, plus the public datasets named in the synopsis (CSIC 2010, CICIDS2017).
5. **Write the paper.**

Research questions

- **RQ1:** Does checking the *final destination* (egress) stop attacks that request-only filtering misses?
- **RQ2:** Does ML add detection beyond fixed rules without too many false alarms?
- **RQ3:** How much delay does the gateway add?
- **RQ4:** Can a restricted AI agent reliably explain the root cause and suggest a fix?

3. Open questions for discussion

1. **Direction.** Is a *working gateway evaluated on a real VM* a valid research contribution, or should the paper centre on measured detection results?
2. **Real VM testing.** Is testing on a real GCP VM acceptable? GCP rejects metadata requests that lack the `Metadata-Flavor: Google` header, so simple attacks may already fail there. Should that be reported as a finding, with testing focused on internal services and on apps that forward headers?
3. **Data.** Is a synthetic dataset acceptable, or is real or public traffic required?
4. **Scope.** Keep the AI-agent part, or concentrate on the two-layer gateway?
5. **Comparison.** Should we compare against existing tools, such as open-source egress proxies like Stripe's Smokescreen and the clouds' built-in metadata protections?
6. **Target.** Which venue (course report, student/workshop paper, or conference) and what timeline fit best?